DATA 221 Homework 6

Chris Low

Problem 1

a).

```
import pandas as pd
import torch

url = "https://vincentarelbundock.github.io/Rdatasets/csv/carData/TitanicSurvival.csv"
df = pd.read_csv(url)

df.head()
```

	rownames	survived	sex	age	passengerClass
0	Allen, Miss. Elisabeth Walton	yes	female	29.0000	1st
1	Allison, Master. Hudson Trevor	yes	male	0.9167	1st
2	Allison, Miss. Helen Loraine	no	female	2.0000	1st
3	Allison, Mr. Hudson Joshua Crei	no	male	30.0000	1st
4	Allison, Mrs. Hudson J C (Bessi	no	female	25.0000	1st

```
# Drop the rownames column and any rows with missing values
df = df.drop(columns=['rownames'])
df = df.dropna()

# Map the 'survived' and 'sex' columns to binary values
df['survived'] = df['survived'].map({'yes': 1, 'no': 0})
df['sex'] = df['sex'].map({'male': 0, 'female': 1})

# Map the 'passengerClass' column to numerical values
```

```
df['passengerClass'] = df['passengerClass'].map({
    '1st': 1,
    '2nd': 2,
    '3rd': 3
})

# Separate our features and target variable
X = df[['age', 'sex', 'passengerClass']]
y = df['survived']

# As mentioned, center the columns of the predictors
X_centered = X - X.mean()

X_tensor = torch.tensor(X_centered.values, dtype=torch.float32)
y_tensor = torch.tensor(y.values, dtype=torch.float32).view(-1, 1)
```

```
print(X_tensor.shape)
print(y_tensor.shape)

print(X_tensor[:5])
print(y_tensor[:5])
```

```
torch.Size([1046, 3])
torch.Size([1046, 1])
tensor([[ -0.8811,   0.6291,  -1.2075],
        [-28.9644,  -0.3709,  -1.2075],
        [-27.8811,   0.6291,  -1.2075],
        [  0.1189,  -0.3709,  -1.2075],
        [ -4.8811,   0.6291,  -1.2075]])
tensor([[1.],
        [1.],
        [0.],
        [0.],
        [0.]])
```

b).

```
def sigmoid(z):
    return 1.0 / (1.0 + torch.exp(-z))
```

```python
def forward_pass(X_in, w):
    z = torch.matmul(X_in, w)
    return sigmoid(z)

# Generate three random weights
torch.manual_seed(42)
weights = torch.randn(3, 1, dtype=torch.float32)

# (29yo, female=1, 1st class=1)
raw_passenger = torch.tensor([29.0, 1.0, 1.0], dtype=torch.float32)

means_tensor = torch.tensor(X.mean().values, dtype=torch.float32)
centered_passenger = raw_passenger - means_tensor

centered_passenger = centered_passenger.view(1, -1)

probability = forward_pass(centered_passenger, weights)

print(f"Probability of survival (29yo, female, 1st class): {probability.item():.4f}")
```

```
Probability of survival (29yo, female, 1st class): 0.3778
```

c). Cross entropy loss and backward pass

```python
def cross_entropy_loss(y_pred, y_true):
    """
    Two-class cross-entropy loss function for binary classification.
    """
    eps = 1e-15
    y_pred = torch.clamp(y_pred, eps, 1 - eps)
    return -torch.mean(y_true * torch.log(y_pred) + (1.0 - y_true) * torch.log(1.0 - y_pred))

test_target = torch.tensor([[1.0]], dtype=torch.float32)  # Given survival
passenger_loss = cross_entropy_loss(probability, test_target)
print(f"Cross Entropy Loss for this passenger: {passenger_loss.item():.4f}")
```

```
Cross Entropy Loss for this passenger: 0.9733
```

```
# Alternatively, we can use pyTorch's built-in loss function: loss.backward()
def backward_pass(X_in, y_pred, y_true):
    # Gradient of binary cross entropy loss with respect to weights w
    N = X_in.shape[0]
    gradient = torch.matmul(X_in.t(), (y_pred - y_true)) / N
    return gradient

gradient = backward_pass(centered_passenger, probability, test_target)
print("Gradient of loss with respect to weights:")
print(gradient)
```

```
Gradient of loss with respect to weights:
tensor([[ 0.5482],
        [-0.3914],
        [ 0.7512]])
```

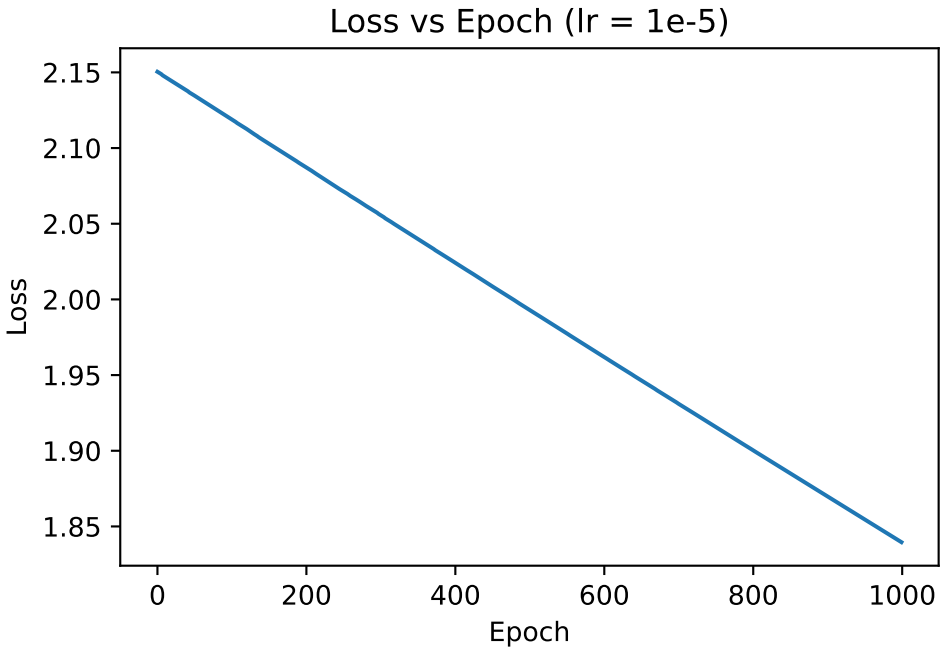
d).

```
import matplotlib.pyplot as plt

learning_rate = 1e-5
epochs = 1000

torch.manual_seed(42)
weights = torch.randn(3, 1, dtype=torch.float32)

loss_history = []
grad_history = []

for epoch in range(epochs):
    y_pred = forward_pass(X_tensor, weights)

    loss = cross_entropy_loss(y_pred, y_tensor)
    grad = backward_pass(X_tensor, y_pred, y_tensor)

    weights = weights - learning_rate * grad

    loss_history.append(loss.item())
    grad_history.append(torch.norm(grad).item())
```

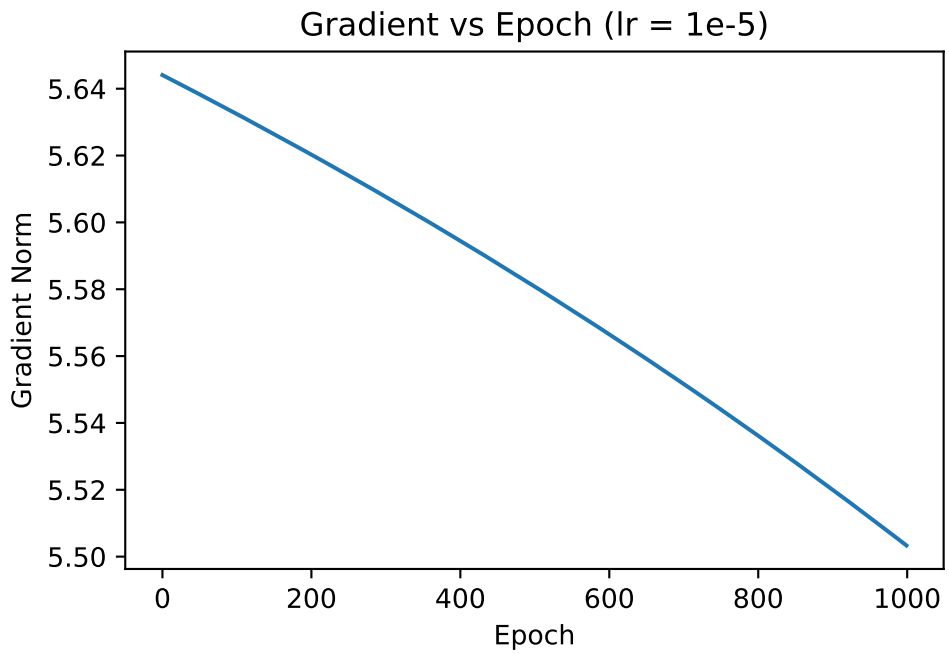
```
print("Final Loss:", loss_history[-1])
```

```
Final Loss: 1.8395979404449463
```

```
# Loss vs Epoch plot
plt.figure()
plt.plot(loss_history)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Loss vs Epoch (lr = 1e-5)")
plt.show()
```



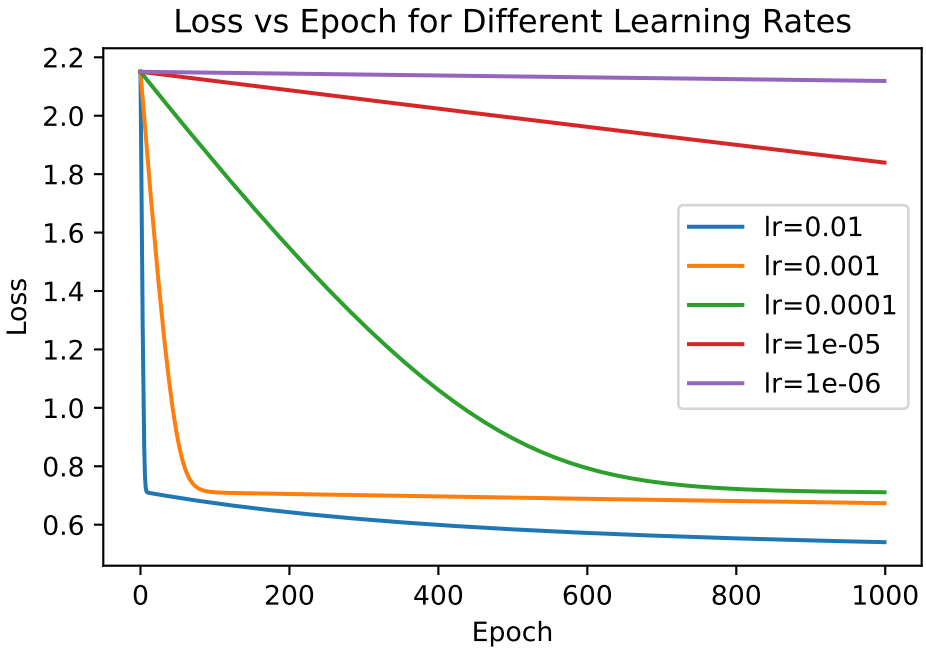

```
# Gradient Norm vs Epoch plot
plt.figure()
plt.plot(grad_history)
plt.xlabel("Epoch")
plt.ylabel("Gradient Norm")
plt.title("Gradient vs Epoch (lr = 1e-5)")
plt.show()
```



e).

```
def train_model(lr, epochs=1000):
    torch.manual_seed(42)
    weights = torch.randn(3, 1, dtype=torch.float32)

    loss_hist_list = []

    for epoch in range(epochs):
        y_pred = forward_pass(X_tensor, weights)
        loss = cross_entropy_loss(y_pred, y_tensor)
        grad = backward_pass(X_tensor, y_pred, y_tensor)
        weights = weights - lr * grad
        loss_hist_list.append(loss.item())

    final_predictions = forward_pass(X_tensor, weights)

    return loss_hist_list, final_predictions

learning_rates = [1e-2, 1e-3, 1e-4, 1e-5, 1e-6]
```

```
res = {}
final_preds = {}

for lr in learning_rates:
    losses, preds = train_model(lr)
    res[lr] = losses
    final_preds[lr] = preds

plt.figure()

for lr in learning_rates:
    plt.plot(res[lr], label=f"lr={lr}")

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Loss vs Epoch for Different Learning Rates")
plt.legend()
plt.show()
```



When comparing the loss curves for the different learning rates, the main difference is how fast the model learns.

With 0.01, the loss drops very quickly and reaches the lowest final value. This learning rate

seems to work the best within 1000 epochs.

With 0.001, the loss also decreases quickly but levels off slightly higher than 0.01.

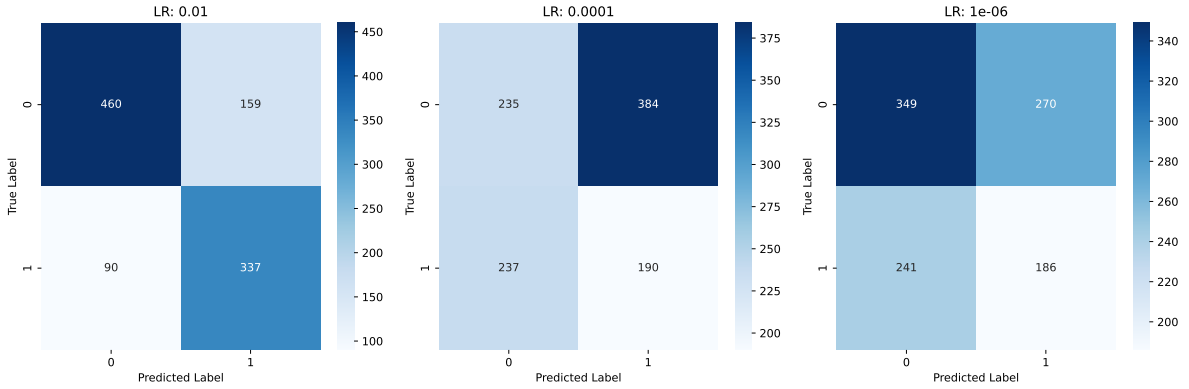
With 0.0001, the loss decreases much more slowly. It improves over time but does not reach as low a value.

With 1e-5 and 1e-6, the loss barely changes. The model is learning very slowly.

Overall, larger learning rates converge faster and reach lower loss values in the same number of epochs. Very small learning rates result in slow training and higher final loss within the fixed 1000 epochs.

f).

```
import seaborn as sns
from sklearn.metrics import confusion_matrix

target_rates = [0.01, 0.0001, 0.000001]
actual_labels = y_tensor.numpy()

fig, ax_row = plt.subplots(1, 3, figsize=(15, 5))

for i in range(len(target_rates)):
    current_lr = target_rates[i]

    raw_probs = final_preds[current_lr].numpy()
    binary_predictions = (raw_probs >= 0.5).astype(int)

    matrix_data = confusion_matrix(actual_labels, binary_predictions)
    sns.heatmap(matrix_data, annot=True, fmt='d', cmap='Blues', ax=ax_row[i])

    ax_row[i].set_title(f"LR: {current_lr}")
    ax_row[i].set_xlabel("Predicted Label")
    ax_row[i].set_ylabel("True Label")

plt.tight_layout()
plt.show()
```



For learning rate 0.01, the model performs the best. It has the highest accuracy (about 76%) and the largest number of true positives and true negatives. It also has fewer false positives and false negatives compared to the other two. This shows that it converged well.

For 0.0001, performance is much worse. The accuracy is only about 41%. The model produces a large number of false positives and false negatives. This suggests it did not converge within 1000 epochs.

For $1e^{-}6$, the accuracy is around 51%, which is slightly better than 0.0001. However, this does not mean it learned more. The model barely trained and behaves closer to a weak baseline classifier. It predicts many negatives, which happens to give slightly better accuracy on this dataset.

Overall, 0.01 clearly gives the best classification results. The smaller learning rates did not converge enough within 1000 epochs.

Problem 2

a).

Note: Dataset preprocessing is mostly the same as in Homework 5, but we will convert the data to PyTorch tensors for this problem.

```
import pandas as pd
import numpy as np
import torch
from sklearn.preprocessing import StandardScaler

epi = pd.read_csv("../hw5/epi_r.csv", index_col='title')

print(f"Original dataset shape: {epi.shape}")


nutritional_vars = ['calories', 'protein', 'fat', 'sodium']

binary_cols = [col for col in epi.columns
               if set(epi[col].dropna().unique()).issubset({0,1})]

# Median imputation for nutritional variables
for col in nutritional_vars:
    epi[col] = epi[col].fillna(epi[col].median())

epi[binary_cols] = epi[binary_cols].fillna(0)

for col in nutritional_vars:
    cap_value = epi[col].quantile(0.99)
    epi[col] = np.where(epi[col] > cap_value, cap_value, epi[col])

# Log transform numeric variables
for col in nutritional_vars:
    epi[col] = np.log1p(epi[col])

min_count = 20
rare_cols = [col for col in binary_cols
             if epi[col].sum() < min_count]

epi = epi.drop(columns=rare_cols)
```

```
print(f"Shape after removing rare tags: {epi.shape}")


y = epi['cake']
X = epi.drop(columns=['cake'])

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_recipe_tensor = torch.tensor(X_scaled, dtype=torch.float32)

print("Final tensor shape:", X_recipe_tensor.shape)
```

```
Original dataset shape: (20052, 679)
Shape after removing rare tags: (20052, 450)
Final tensor shape: torch.Size([20052, 449])
```

b).

```
import torch.nn as nn

class AutoEncoder(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.BatchNorm1d(256),
            nn.ReLU(),
            nn.Linear(256, 10)
        )
        self.decoder = nn.Sequential(
            nn.Linear(10, 256),
            nn.ReLU(),
            nn.Linear(256, input_dim)
        )

    def forward(self, x):
        encoded = self.encoder(x)
        decoded = self.decoder(encoded)
        return decoded, encoded
```

```
input_dim = X_recipe_tensor.shape[1]
model = AutoEncoder(input_dim)
print(model)
```

```
AutoEncoder(
  (encoder): Sequential(
    (0): Linear(in_features=449, out_features=256, bias=True)
    (1): BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): Linear(in_features=256, out_features=10, bias=True)
  )
  (decoder): Sequential(
    (0): Linear(in_features=10, out_features=256, bias=True)
    (1): ReLU()
    (2): Linear(in_features=256, out_features=449, bias=True)
  )
)
```

c).

```
import torch.nn as nn
import torch.optim as optim

input_dim = X_recipe_tensor.shape[1]

model = AutoEncoder(input_dim)
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

d).

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset

X_recipe_tensor = X_recipe_tensor.to(torch.float32)
```

```
print("Tensor shape:", X_recipe_tensor.shape)

epochs = 100
batch_size = 256
learning_rate = 0.001

dataset = TensorDataset(X_recipe_tensor)
dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True)

input_dim = X_recipe_tensor.shape[1]
model = AutoEncoder(input_dim)

criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate)

loss_history = []

model.train()

for epoch in range(epochs):
    epoch_loss = 0.0

    for batch in dataloader:
        X_batch = batch[0]

        optimizer.zero_grad()

        reconstructed, encoded = model(X_batch)

        loss = criterion(reconstructed, X_batch)

        loss.backward()
        optimizer.step()

        epoch_loss += loss.item()

    epoch_loss /= len(dataloader)
    loss_history.append(epoch_loss)

    print(f"Epoch [{epoch+1}/{epochs}] - Loss: {epoch_loss:.6f}")
```

```
Tensor shape: torch.Size([20052, 449])
Epoch [1/100] - Loss: 0.946628
Epoch [2/100] - Loss: 0.878483
Epoch [3/100] - Loss: 0.842629
Epoch [4/100] - Loss: 0.822080
Epoch [5/100] - Loss: 0.807718
Epoch [6/100] - Loss: 0.798081
Epoch [7/100] - Loss: 0.786779
Epoch [8/100] - Loss: 0.778604
Epoch [9/100] - Loss: 0.769853
Epoch [10/100] - Loss: 0.761579
Epoch [11/100] - Loss: 0.755094
Epoch [12/100] - Loss: 0.745919
Epoch [13/100] - Loss: 0.740095
Epoch [14/100] - Loss: 0.733719
Epoch [15/100] - Loss: 0.728326
Epoch [16/100] - Loss: 0.722630
Epoch [17/100] - Loss: 0.717385
Epoch [18/100] - Loss: 0.711898
Epoch [19/100] - Loss: 0.707472
Epoch [20/100] - Loss: 0.703535
Epoch [21/100] - Loss: 0.700213
Epoch [22/100] - Loss: 0.697086
Epoch [23/100] - Loss: 0.691932
Epoch [24/100] - Loss: 0.689484
Epoch [25/100] - Loss: 0.687118
Epoch [26/100] - Loss: 0.683124
Epoch [27/100] - Loss: 0.680232
Epoch [28/100] - Loss: 0.679386
Epoch [29/100] - Loss: 0.677101
Epoch [30/100] - Loss: 0.675314
Epoch [31/100] - Loss: 0.672826
Epoch [32/100] - Loss: 0.670013
Epoch [33/100] - Loss: 0.668900
Epoch [34/100] - Loss: 0.666428
Epoch [35/100] - Loss: 0.665284
Epoch [36/100] - Loss: 0.664795
Epoch [37/100] - Loss: 0.662381
Epoch [38/100] - Loss: 0.660280
Epoch [39/100] - Loss: 0.659010
Epoch [40/100] - Loss: 0.658503
Epoch [41/100] - Loss: 0.656378
Epoch [42/100] - Loss: 0.655307
```

```
Epoch [43/100] - Loss: 0.654620
Epoch [44/100] - Loss: 0.653591
Epoch [45/100] - Loss: 0.650973
Epoch [46/100] - Loss: 0.650738
Epoch [47/100] - Loss: 0.649239
Epoch [48/100] - Loss: 0.648591
Epoch [49/100] - Loss: 0.647793
Epoch [50/100] - Loss: 0.646801
Epoch [51/100] - Loss: 0.645064
Epoch [52/100] - Loss: 0.645117
Epoch [53/100] - Loss: 0.644466
Epoch [54/100] - Loss: 0.643679
Epoch [55/100] - Loss: 0.642107
Epoch [56/100] - Loss: 0.641044
Epoch [57/100] - Loss: 0.640407
Epoch [58/100] - Loss: 0.639757
Epoch [59/100] - Loss: 0.639614
Epoch [60/100] - Loss: 0.638355
Epoch [61/100] - Loss: 0.637109
Epoch [62/100] - Loss: 0.637659
Epoch [63/100] - Loss: 0.635366
Epoch [64/100] - Loss: 0.636346
Epoch [65/100] - Loss: 0.635396
Epoch [66/100] - Loss: 0.634427
Epoch [67/100] - Loss: 0.633304
Epoch [68/100] - Loss: 0.632381
Epoch [69/100] - Loss: 0.634055
Epoch [70/100] - Loss: 0.631928
Epoch [71/100] - Loss: 0.631273
Epoch [72/100] - Loss: 0.630649
Epoch [73/100] - Loss: 0.629222
Epoch [74/100] - Loss: 0.628174
Epoch [75/100] - Loss: 0.628493
Epoch [76/100] - Loss: 0.628166
Epoch [77/100] - Loss: 0.627373
Epoch [78/100] - Loss: 0.626508
Epoch [79/100] - Loss: 0.626848
Epoch [80/100] - Loss: 0.626501
Epoch [81/100] - Loss: 0.626148
Epoch [82/100] - Loss: 0.624440
Epoch [83/100] - Loss: 0.625258
Epoch [84/100] - Loss: 0.625229
Epoch [85/100] - Loss: 0.624972
```

```
Epoch [86/100] - Loss: 0.624142
Epoch [87/100] - Loss: 0.623196
Epoch [88/100] - Loss: 0.623471
Epoch [89/100] - Loss: 0.621900
Epoch [90/100] - Loss: 0.622199
Epoch [91/100] - Loss: 0.621782
Epoch [92/100] - Loss: 0.621619
Epoch [93/100] - Loss: 0.620605
Epoch [94/100] - Loss: 0.620418
Epoch [95/100] - Loss: 0.620571
Epoch [96/100] - Loss: 0.620219
Epoch [97/100] - Loss: 0.620163
Epoch [98/100] - Loss: 0.619644
Epoch [99/100] - Loss: 0.618841
Epoch [100/100] - Loss: 0.618784
```

e).

```
model.eval()

with torch.no_grad():
    _, latent = model(X_recipe_tensor)

latent = latent.numpy()

print(latent.shape)
```

```
(20052, 10)
```

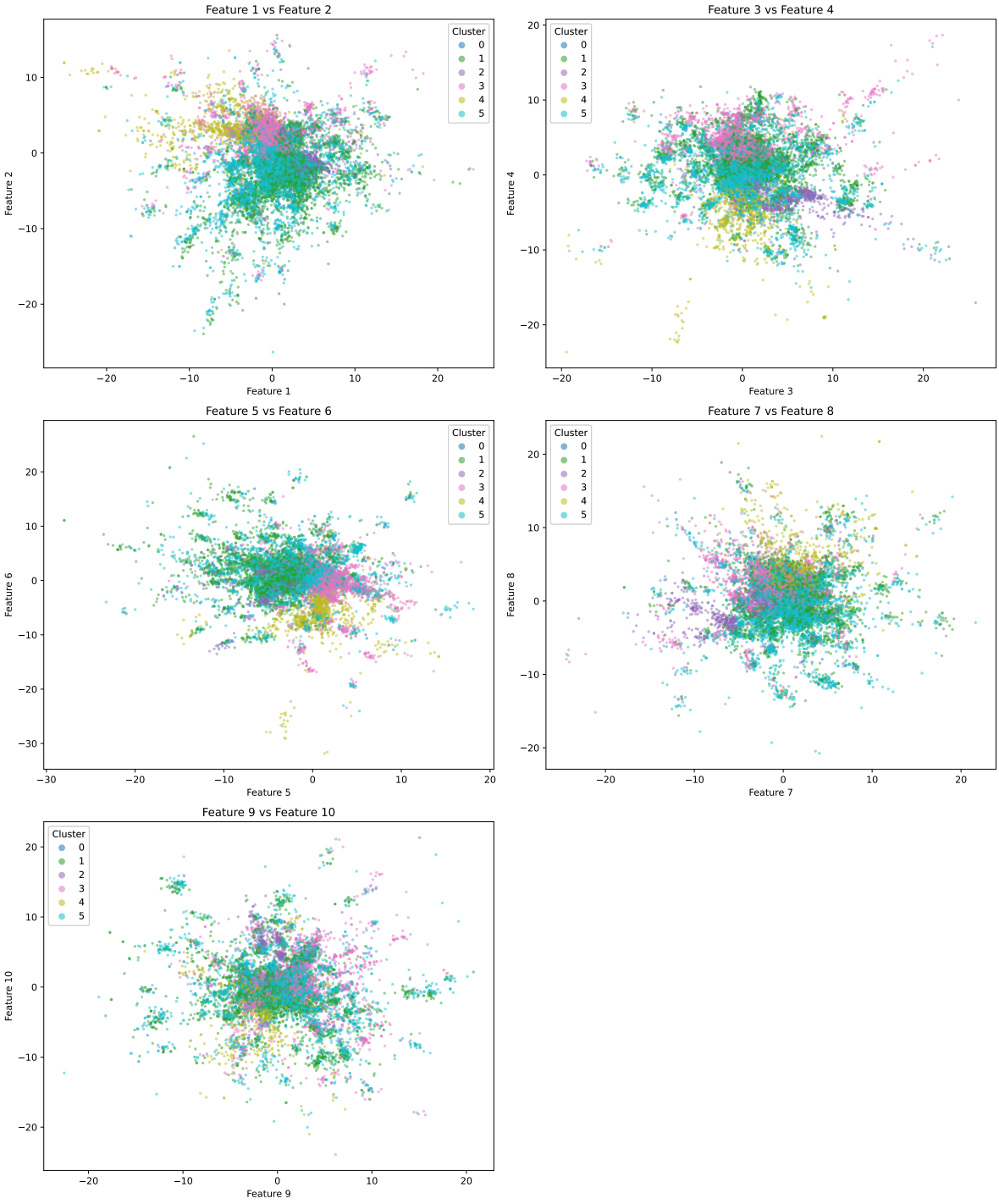
Re-create cluster labels:

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

X_full = X.copy()

scaler_full = StandardScaler()
X_full_scaled = scaler_full.fit_transform(X_full)

# From HW5, we found that K=6 is the best choice based on the elbow method and silhouette sc
```

```
best_K = 6

kmeans_final = KMeans(
    n_clusters=best_K,
    random_state=42,
    n_init=20
)

cluster_labels = kmeans_final.fit_predict(X_full_scaled)

print("Cluster labels shape:", cluster_labels.shape)
```

```
Cluster labels shape: (20052,)
```

```
import matplotlib.pyplot as plt

feature_pairs = [(0,1), (2,3), (4,5), (6,7), (8,9)]

fig, axes = plt.subplots(3, 2, figsize=(15,18))
axes = axes.flatten()

for i, (f1, f2) in enumerate(feature_pairs):
    scatter = axes[i].scatter(
        latent[:, f1],
        latent[:, f2],
        c=cluster_labels,
        cmap="tab10",
        alpha=0.5,
        s=4
    )

    axes[i].set_xlabel(f"Feature {f1+1}")
    axes[i].set_ylabel(f"Feature {f2+1}")
    axes[i].set_title(f"Feature {f1+1} vs Feature {f2+1}")

    legend = axes[i].legend(*scatter.legend_elements(), title="Cluster")
    axes[i].add_artist(legend)

fig.delaxes(axes[5])

plt.tight_layout()
plt.show()
```



In Homework 5, the PCA plots, especially PC1 vs PC2, showed clearer separation between some of the clusters. Certain groups stretched in different directions, and a few clusters were

visibly more distinct along the first principal components.

In contrast, the autoencoder feature plots look more mixed. While there are slight tendencies for some clusters to lean in certain directions (for example, cluster 3 leans towards the top left in feature 1-2 and feature 3-4), the separation is not nearly as clean in any single two-dimensional projection. Most of the points overlap heavily.

This makes sense because PCA is explicitly finding directions of maximum variance, which can exaggerate separation along the first few components. The autoencoder, however, is trained to reconstruct the data rather than to spread it out. So, it compresses recipes into a dense latent space where cluster boundaries are less visually obvious.